



BOARD OF INTERMEDIATE & SECONDARY EDUCATION, LARKANO

Established to serve affiliated schools and colleges across the Larkano region

Higher Secondary School Certificate
Examination Syllabus

COMPUTER ALGORITHMS

Grades XI – XII

This syllabus will be examined in both Annual and Re-sit Examination sessions beginning with the Annual Examinations 2023.

Published by

Board of Intermediate & Secondary Education, Larkano
Curriculum & Examination Development Unit
Larkano, Sindh, Pakistan
Latest revision: 2023

© Board of Intermediate & Secondary Education, Larkano, 2023
All rights and entitlements reserved. This syllabus is developed by BISE Larkano for distribution to its affiliated schools and colleges only, and may not be reproduced without prior written permission.

Table of Contents

Preface.....	3
Understanding of BISE Larkano Syllabi.....	4
Subject Rationale.....	5
Concept Map.....	6
Student Learning Outcomes – Part I (Grade XI).....	7
Student Learning Outcomes – Part II (Grade XII).....	9
Scheme of Assessment.....	11
Academic Integrity and Responsible Use of Technology.....	14
Acknowledgements.....	15

Preface

The Board of Intermediate & Secondary Education, Larkano (BISE Larkano) is a public examination body responsible for the conduct of Secondary School Certificate (SSC) and Higher Secondary School Certificate (HSSC) examinations for its affiliated schools and colleges. The Board is committed to developing examination syllabi that promote conceptual understanding and higher-order thinking, and that remain aligned with the national curriculum and with contemporary international standards in computing education.

This document sets out the syllabus for Computer Algorithms at the HSSC level (Grades XI and XII). It has been developed in response to the growing importance of algorithmic thinking, computational problem-solving and programming practice as foundational skills for further study and employment in computing and related fields.

The syllabus was developed following consultation with subject specialists, teachers of affiliated institutions, and reviewers with expertise in computer science education, and reflects a deliberate effort to move examination preparation away from rote memorisation of code and toward the disciplined analysis, design and honest evaluation of algorithms.

We are confident that this syllabus will support teachers and students of affiliated institutions in building a rigorous and practical understanding of algorithms, and we wish every success to those who study and teach it.

Controller of Examinations

Board of Intermediate & Secondary Education, Larkano

Understanding of BISE Larkano Syllabi

1. The BISE Larkano syllabi guide students, teachers, parents and other stakeholders regarding the topics that will be taught and examined in each grade (XI and XII). In each syllabus document, the content progresses from simple to complex, facilitating gradual, conceptual learning.
2. The topics of each syllabus are divided into sub-topics and Student Learning Outcomes (SLOs). The sub-topics and SLOs define the depth and breadth at which each topic will be taught, learnt and examined.
3. Each SLO begins with an achievable, assessable command word such as ‘describe’, ‘apply’, ‘analyse’ or ‘evaluate’. Examination questions are framed using the same command words, or their equivalents, to elicit evidence of these competencies in students’ responses.
4. The SLOs are classified under three cognitive levels: Knowledge (K), Understanding (U), and Application and other higher-order skills (A), to support effective planning by teachers and the balanced construction of examination papers.
5. By focusing on the achievement of SLOs rather than on the reproduction of memorised code, this syllabus aims to encourage students to reason about algorithms, to justify their choices, and to communicate their thinking clearly — skills that extend well beyond a single assignment or examination.
6. While the syllabus recommends locally available textbooks and resources for achieving these outcomes, teachers and students are encouraged to draw on multiple teaching and learning resources, and to acknowledge any resource that materially shapes a solution submitted for assessment.

Subject Rationale of Computer Algorithms

What will you learn in Computer Algorithms?

Algorithms are the reasoned, step-by-step procedures that underlie every piece of software students will ever use or build. Computer Algorithms teaches students to analyse the correctness and efficiency of a procedure using methods that are mathematically rigorous, rather than relying on intuition or trial and error.

Many students who have written working programs have not yet learned to ask whether a program is efficient, whether it is correct on every input, or whether a faster solution is even possible. This syllabus addresses that gap directly. Students will study several general approaches to algorithm design — divide-and-conquer, dynamic programming, the greedy approach and backtracking — and will learn to express, trace and analyse algorithms using structured pseudocode before implementing them in Python.

The syllabus also introduces students to the theory of computational complexity and NP-completeness, giving them the vocabulary to recognise when a problem is fundamentally hard, and the ability to explain — convincingly, and to a non-specialist — why a good, efficient solution should not be expected.

Where will it take you?

The AKU-EB-style foundation provided by this syllabus prepares students for further study in Computer Science, Software Engineering, Data Science and related disciplines, and for a wide range of careers, including:

- Software Engineer
- Data Scientist
- Systems Analyst
- Research Scientist
- Cybersecurity Analyst
- Machine Learning Engineer
- Database Administrator
- Competitive Programmer / Algorithm Researcher

How to approach the syllabus?

The concept map on the following page gives an overview of the entire syllabus. The topics and Student Learning Outcomes guide what has to be achieved, and the Scheme of Assessment explains how each topic will be weighted and examined.

Concept Map

The concept map below organises the syllabus into four connected strands, all resting on a foundation of Reasoning and Problem Decomposition — the discipline of reducing a real problem to a simpler one that can be solved and then generalised.

Design Strategies

- Divide and Conquer
- Dynamic Programming
- The Greedy Approach
- Backtracking

Analysis of Algorithms

- Time and Space Complexity
- Asymptotic (Big-O) Notation
- Recurrence Relations
- Correctness and Loop Invariants

Structures and Traversal

- Arrays, Lists, Stacks, Queues
- Trees and Tree Traversal
- Graphs, Search and Shortest Paths
- Sorting Algorithms

Complexity Theory and Practice

- Classes P and NP
- NP-Completeness
- Python Implementation and Testing
- Communicating Algorithmic Arguments

Reasoning and Problem Decomposition underpins all four strands: every design strategy, every analysis technique and every data structure in this syllabus is ultimately a tool for reducing a complex problem to one that can be solved with confidence and explained with clarity.

Student Learning Outcomes of Computer Algorithms

Part I (Grade XI)

1. Foundations of Algorithm Analysis

Topics and Sub-topics	Student Learning Outcomes	K	U	A
1.1 Algorithms and Problem Solving	define an algorithm and describe its essential properties, i.e. finiteness, definiteness and effectiveness;	✓		
	distinguish between a problem statement and an algorithm that solves it;		✓	
	express an algorithm as structured pseudocode;			✓
	trace, by hand, the execution of an algorithm given in pseudocode for a specific input;			✓
1.2 Data Structures Underlying Algorithms	review the structure and typical operations of arrays, linked lists, stacks and queues;	✓		
	review the structure and traversal strategies of trees and graphs;	✓		
	select an appropriate data structure to support a given algorithmic task;			✓

2. Growth of Functions and Efficiency Analysis

Topics and Sub-topics	Student Learning Outcomes	K	U	A
2.1 Measuring Efficiency	distinguish between the time complexity and the space complexity of an algorithm;		✓	
	distinguish between best-case, average-case and worst-case analysis of an algorithm;		✓	
2.2 Asymptotic Notation	define Big-O, Big-Omega and Big-Theta notation;	✓		
	define little-o notation;	✓		
	determine the order of growth of a given function using asymptotic notation;			✓
	compare the relative growth rates of common functions, i.e. constant, logarithmic, linear, linearithmic, polynomial and exponential;			✓
	analyse the time complexity of an algorithm expressed in pseudocode;			✓

3. The Divide-and-Conquer Strategy

Topics and Sub-topics	Student Learning Outcomes	K	U	A
3.1 Principles of Divide and Conquer	describe the divide, conquer and combine phases of a divide-and-conquer algorithm;	✓		
	apply the divide-and-conquer approach to design an algorithm for a given problem;			✓
3.2 Analysing Divide-and-Conquer Algorithms	formulate a recurrence relation describing the running time of a divide-and-conquer algorithm;			✓
	solve simple recurrence relations to determine the time complexity of an algorithm;			✓

4. The Greedy Approach

Topics and Sub-topics	Student Learning Outcomes	K	U	A
4.1 Principles of Greedy Algorithms	describe the greedy-choice property and the concept of optimal substructure;	✓		
	apply a greedy strategy to design an algorithm for a given problem;			✓
	evaluate whether a greedy strategy produces an optimal solution for a given problem;			✓

5. Backtracking

Topics and Sub-topics	Student Learning Outcomes	K	U	A
5.1 Principles of Backtracking	describe backtracking as a systematic method for exploring a space of candidate solutions;	✓		
	apply backtracking to design an algorithm for a given constraint-satisfaction problem;			✓
	use pruning to improve the efficiency of a backtracking algorithm;			✓

Part II (Grade XII)

6. Dynamic Programming

Topics and Sub-topics	Student Learning Outcomes	K	U	A
6.1 Principles of Dynamic Programming	describe the concepts of overlapping subproblems and optimal substructure;	✓		
	distinguish between the top-down (memoisation) and bottom-up (tabulation) approaches to dynamic programming;		✓	
	apply dynamic programming to design an algorithm for a given problem;			✓
	analyse the time and space complexity of a dynamic programming solution;			✓

7. Tree and Graph Algorithms

Topics and Sub-topics	Student Learning Outcomes	K	U	A
7.1 Tree Algorithms	apply pre-order, in-order, post-order and level-order traversal to a binary or general tree;			✓
7.2 Graph Algorithms	describe representations of a graph, i.e. adjacency list and adjacency matrix;	✓		
	apply depth-first search and breadth-first search to a given graph;			✓
	apply an algorithm for finding a minimum spanning tree of a weighted graph;			✓
	apply an algorithm for finding shortest paths in a weighted graph;			✓

8. Sorting and Correctness of Algorithms

Topics and Sub-topics	Student Learning Outcomes	K	U	A
8.1 Comparison-based Sorting	apply divide-and-conquer sorting algorithms, e.g. merge sort and quicksort, to a given input;			✓
	compare the time complexity of different sorting algorithms;		✓	
8.2 Reasoning about Correctness	use a loop invariant to reason about the correctness of an iterative algorithm;			✓

9. Computational Complexity and NP-Completeness

Topics and Sub-topics	Student Learning Outcomes	K	U	A
9.1 Complexity Classes	describe the complexity classes P and NP;	✓		
	describe the concept of a polynomial-time reduction between problems;	✓		
	describe what it means for a problem to be NP-complete;		✓	
9.2 Reasoning about Intractability	explain, with justification, why an efficient deterministic polynomial-time solution to an NP-complete problem is not expected to exist;			✓
	communicate an argument for the intractability of a problem in terms accessible to a non-specialist audience;			✓

10. Algorithm Implementation in Python

Topics and Sub-topics	Student Learning Outcomes	K	U	A
10.1 Translating Algorithms into Code	implement a pseudocode algorithm as a working Python program;			✓
	test a Python implementation of an algorithm against a range of inputs, including edge cases and general, not merely specific, instances;			✓
	debug and correct errors in a Python implementation of an algorithm;			✓
10.2 Programming Practice and Documentation	document Python code with comments that explain the underlying algorithm and cite any external source consulted;			✓

Scheme of Assessment

Each grade of Computer Algorithms is examined through a Theory paper and a Practical (laboratory) examination, whose marks combine to the 100 marks recorded for the subject on the BISE Larkano Marks Certificate. The Theory paper carries 85 marks and the Practical examination carries 15 marks, matching the maximum marks shown against ‘Computer Algorithms (TH)’ and ‘Computer Algorithms (PR)’ for each year on the certificate.

Grade XI

Table 1: Number of Student Learning Outcomes by Cognitive Level

Topic No.	Topic	No. of Sub-topics	K	U	A	Total
1	Foundations of Algorithm Analysis	2	3	1	3	7
2	Growth of Functions and Efficiency Analysis	2	2	2	3	7
3	The Divide-and-Conquer Strategy	2	1	0	3	4
4	The Greedy Approach	1	1	0	2	3
5	Backtracking	1	1	0	2	3
	Total	8	8	3	13	24
	Percentage		33%	13%	54%	100%

Table 2: Theory Exam Specification — Grade XI — Computer Algorithms (TH)-I — 85 Marks

Topic No.	Topic	MCQs (Marks)	CRQs (Marks)	Total Marks
1	Foundations of Algorithm Analysis	5	10 (1 CRQ)	15
2	Growth of Functions and Efficiency Analysis	7	13 (1 CRQ)	20
3	The Divide-and-Conquer Strategy	6	11 (1 CRQ)	17
4	The Greedy Approach	5	10 (1 CRQ)	15
5	Backtracking	7	11 (1 CRQ)	18
	Total	30	55	85

Table 2A: Overall Marks Distribution — Grade XI

Component	Certificate Entry	Marks
Theory Examination	Computer Algorithms (TH)-I	85
Practical Examination	Computer Algorithms (PR)-I	15
Total		100

Grade XII

Table 3: Number of Student Learning Outcomes by Cognitive Level

Topic No.	Topic	No. of Sub-topics	K	U	A	Total
6	Dynamic Programming	1	1	1	2	4
7	Tree and Graph Algorithms	2	1	0	4	5
8	Sorting and Correctness of Algorithms	2	0	1	2	3
9	Computational Complexity and NP-Completeness	2	2	1	2	5
10	Algorithm Implementation in Python	2	0	0	4	4
	Total	9	4	3	14	21
	Percentage		19%	14%	67%	100%

Table 4: Theory Exam Specification — Grade XII — Computer Algorithms (TH)-II — 85 Marks

Topic No.	Topic	MCQs (Marks)	CRQs (Marks)	Total Marks
6	Dynamic Programming	6	12 (1 CRQ)	18
7	Tree and Graph Algorithms	7	12 (1 CRQ)	19
8	Sorting and Correctness of Algorithms	5	9 (1 CRQ)	14
9	Computational Complexity and NP-Completeness	6	10 (1 CRQ)	16
10	Algorithm Implementation in Python	6	12 (1 CRQ)	18
	Total	30	55	85

Table 4A: Overall Marks Distribution — Grade XII

Component	Certificate Entry	Marks
Theory Examination	Computer Algorithms (TH)-II	85
Practical Examination	Computer Algorithms (PR)-II	15
Total		100

- A Multiple Choice Question (MCQ) requires candidates to choose one best/correct answer from four options. Each MCQ carries ONE mark.
- A Constructed Response Question (CRQ) requires students to respond with structured pseudocode, worked analysis, a short proof/argument, or a Python code fragment, as appropriate to the SLO being assessed.
- There will be two Theory examinations, one at the end of Grade XI (Computer Algorithms (TH)-I) and one at the end of Grade XII (Computer Algorithms (TH)-II). Each Theory paper carries 85 marks, is of 3 hours' duration, and consists of Paper I (MCQs) and Paper II (CRQs).
- There will be two Practical examinations, one at the end of Grade XI (Computer Algorithms (PR)-I) and one at the end of Grade XII (Computer Algorithms (PR)-II). Each Practical examination carries 15 marks and requires students to implement, test and debug a short algorithm in Python under supervised conditions.

- The Theory and Practical marks for a given year are reported separately on the BISE Larkano Marks Certificate and are combined only in the year's subject total ($85 + 15 = 100$). A candidate must attempt both components; a Theory or Practical mark cannot substitute for the other.
- Tables 1 and 3 indicate the number and nature of SLOs in each topic and are provided as a guide to the construction of examination papers; they do not translate directly into marks. Emphasis has deliberately been placed on Application and other higher-order skills, to discourage rote memorisation of code.
- Where a CRQ requires a Python code fragment, partial credit is available for a solution that is correct in approach but contains minor implementation errors, provided the underlying algorithm is sound.

Academic Integrity and Responsible Use of Technology

All work submitted for assessment in Computer Algorithms — whether pseudocode, a written analysis, or a Python program — must represent the individual, honest effort of the student submitting it. Affiliated institutions are expected to uphold the BISE Larkano Academic Integrity Policy in full, and to make students aware of the following subject-specific guidance.

Collaboration

Students are encouraged to discuss algorithmic concepts, general strategies and approaches with their peers, and to consult the prescribed textbook and other reputable resources to build their understanding. However, copying a solution from any source — including another student, a book, or an online repository — and submitting it as one's own, with or without superficial changes, is a serious violation of academic integrity.

Use of Artificial Intelligence and Other Generative Tools

Affiliated institutions may permit or restrict the use of AI and other generative tools in coursework at their own discretion, consistent with their local policy; this syllabus does not itself mandate a single institutional position. Where an institution permits limited use of such tools for understanding a concept, any resulting influence on a submitted solution must be disclosed and cited, in the same way as any other external source. Under no circumstances may a student submit algorithmic analysis or code that they cannot explain and defend in their own words if asked to do so by an examiner or teacher.

Correctness Over Convenience

An algorithm, and any accompanying implementation, is expected to work correctly on every valid input, not merely on the specific examples used to illustrate or test it during development. Solutions that are narrowly tailored to pass a small set of sample tests, without representing a general and correct solution to the stated problem, will not receive full credit.

Acknowledgements

The Board of Intermediate & Secondary Education, Larkano gratefully acknowledges the contributions of all those who took part in the development of this Computer Algorithms syllabus, including the subject specialists, affiliated-school teachers, and higher-education reviewers who generously gave their time, expertise and feedback throughout the review process.

The Board also thanks the students and teachers of affiliated institutions across the Larkano region who participated in the needs-assessment consultation that informed this syllabus, and the internal curriculum, assessment and administrative teams who supported its preparation and publication.